



React Native Master Guide — Level 5 (Complete Verbatim Edition)

Centralized API Architecture, Axios Interceptors, Hardware-Backed Secure Auth & NetInfo Offline Resilience

EXPO V54 • NATIEWIND / TAILWIND CSS • ES6+



Level 5.1: Centralized API Architecture & Axios Interceptors

1. Mental Model: Web Fetch vs. Mobile Network Reality

Web apps usually run on stable high-speed broadband. Kani mobile apps lo user **car lo velthunte 5G nundi 4G/2G ki drop avthundi, leda lift/tunnel loki vellinappudu offline aypothundi.**

Prathi component lo direct ga `fetch('https://api.example.com/...')` rayadam mobile apps lo bad practice. Instead, manaki **Centralized API Client with Interceptors** kavali:



2. Why Axios Interceptors are a Mobile Lifesaver?

- Request Interceptor:** Prathi API call ki manual ga header rayalsina pani lekunda, automatic ga storage nundi `JWT Token` theeskuni `headers.Authorization = 'Bearer token'` ga attach chethundi.
- Response Interceptor:** Backend nundi `401 Unauthorized` (Token expired) error vachinappudu, global ga user ni logout chesi Login screen ki redirect chethundi.
- Timeout Handling:** Mobile network slow unte `timeout: 10000` (10 seconds) lo request cancel chesi clean ga *"Slow connection, please retry"* error isthundi.

Code Example: Production Centralized API Client & User Feed

```
import React, { useState, useEffect } from 'react';
import { View, Text, FlatList, ActivityIndicator, Pressable } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// =====
// 🌐 1. CENTRALIZED API SERVICE (Simulated Axios Client)
// =====
const apiClient = {
  get: async (endpoint) => {
    // Request Interceptor: Injects auth token
    const token = "mock_jwt_token_xyz89";
    console.log(`[API REQUEST] 🚀 Calling: ${endpoint} | Auth Token: Bearer ${token}`);

    // Simulating Real Network Delay & Public JSON API
    const response = await fetch(`https://jsonplaceholder.typicode.com${endpoint}`, {
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json',
      },
    });

    if (!response.ok) {
      throw new Error(`HTTP Error: ${response.status}`);
    }

    return await response.json();
  },
};

// =====
// 🏠 2. MAIN COMPONENT: Live Network Feed
// =====
const App = () => {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Function to fetch data from API
  const fetchUsers = async () => {
    try {
      setLoading(true);
      setError(null);

      // Centralized API call
      const data = await apiClient.get('/users');
      setUsers(data);
    } catch (err) {
      setError(err.message || 'Something went wrong. Check network.');
```

```

<View className="flex-1 justify-center items-center gap-3">
  <ActivityIndicator size="large" color="#38bdf8" />
  <Text className="text-slate-400 text-xs font-semibold tracking-wider">
    FETCHING LIVE USERS...
  </Text>
</View>
) : error ? (
  /* 2. ERROR STATE & RETRY BUTTON */
  <View className="flex-1 justify-center items-center px-6">
    <View className="w-14 h-14 rounded-2xl bg-red-500/20 border border-red-500/30 items-center justify-center mb-3">
      <Text className="text-2xl">⚠️</Text>
    </View>
    <Text className="text-white font-bold text-base mb-1">Network Request Failed</Text>
    <Text className="text-slate-400 text-xs text-center mb-6">{error}</Text>
    <Pressable
      onPress={fetchUsers}
      className="bg-sky-500 active:bg-sky-600 px-6 py-3 rounded-xl items-center w-full"
    >
      <Text className="text-white font-bold text-xs">Try Again 🔄</Text>
    </Pressable>
  </View>
) : (
  /* 3. SUCCESS DATA LIST */
  <FlatList
    data={users}
    keyExtractor={(item) => item.id.toString()}
    ItemSeparatorComponent={() => <View className="h-3" />}
    renderItem={({ item }) => (
      <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row justify-between items-center">
        <View className="flex-1 pr-3">
          <Text className="text-white font-bold text-base">{item.name}</Text>
          <Text className="text-sky-400 text-xs mt-0.5">@{item.username.toLowerCase()}</Text>
          <Text className="text-slate-400 text-xs mt-1">📧 {item.email}</Text>
        </View>
        <View className="bg-slate-950 px-3 py-1.5 rounded-xl border border-slate-800">
          <Text className="text-slate-400 text-[11px] font-mono">{item.address.city}</Text>
        </View>
      </View>
    )}
    showsVerticalScrollIndicator={false}
    contentContainerStyle={{ paddingBottom: 24 }}
  />
)}
</SafeAreaView>
</SafeAreaProvider>
);
};

export default App;

```

🧑 Telugu + English Explanation:

- **Centralized API Client:** Manam app lo 20 ververu screens nundi API calls cheyalsi vasthe, prathi screen lo `fetch` raasi headers pettadam bad architecture. Okka `apiClient.js` file lo setup chesthe, token headers and error logging anni automatic ga okka chote manage avthayi.
- **Loading & Error States:** Mobile apps lo slow network valla API call `2-3 seconds` theskovachu. User ki blank screen chupinchakunda `ActivityIndicator` spinner chupinchali. Request fail ayithe "Try Again" retry button ivvali.

🔍 Quick Check:

Q: Where is the best place to catch a 401 Unauthorized globally and redirect to Login?

A: In the **Centralized Axios Response Interceptor**. You write the logout logic once, and all 50+ screens in your app are automatically protected!

 Level 5.2: Secure Token Storage & Persistent Auth Flow

1. Mental Model: Why Plain Storage is Dangerous in Mobile

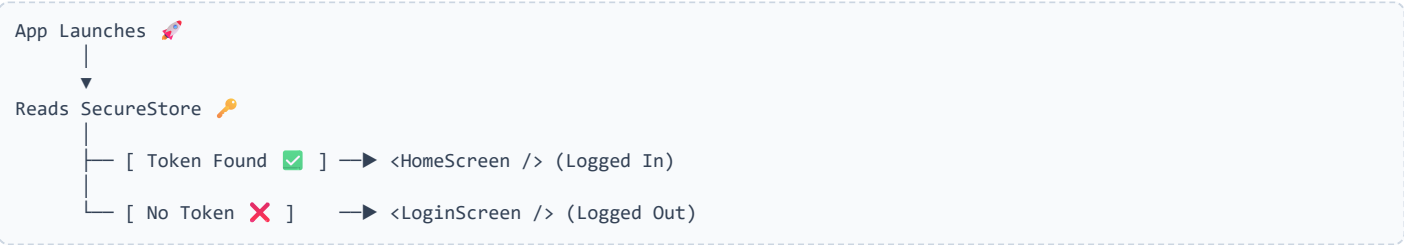
Web lo developer lu token ni `localStorage` lo save chestharu. Mobile apps lo kooda `AsyncStorage` untundi, **kani sensitive data (JWT tokens, passwords, credit card info) ni plain `AsyncStorage` lo eppudu pettakoodadu! ❌**

Storage Type	iOS Implementation	Android Implementation	Encryption	Best Used For
<code>AsyncStorage</code>	Plain text file	SQLite / XML file	❌ None (Plaintext)	UI Theme, language preference, draft text
<code>expo-secure-store</code>	iOS Keychain	Android Keystore (AES-256)	✅ Hardware Encrypted	JWT Auth Tokens, Refresh Tokens, PINs

2. The Conditional Navigation Auth Flow (No Flickers!)

Mobile apps lo standard pattern:

- 1. App open avvagane hardware SecureStore nundi token check chesthundi (`isLoading = true`).
- 2. **Token unte** → Direct ga `<HomeScreen>` / Dashboard render avthundi.
- 3. **Token lekapothe** → `<LoginScreen>` render avthundi.



Code Example: Full Persistent Authentication Flow

```
import React, { useState, useEffect } from 'react';
import { View, Text, TextInput, Pressable, ActivityIndicator, Keyboard } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// =====
// 🛡️ 1. MOCK SECURE STORAGE SERVICE (Simulating expo-secure-store)
// =====
let mockDeviceKeystore = null; // Simulates hardware encrypted storage

const SecureStorage = {
  saveToken: async (token) => {
    mockDeviceKeystore = token;
  },
  getToken: async () => {
    // Simulating slight hardware read delay
    await new Promise((res) => setTimeout(res, 600));
    return mockDeviceKeystore;
  },
  deleteToken: async () => {
    mockDeviceKeystore = null;
  },
};

// =====
// 📺 2. SCREEN A: Login Screen (Auth Stack)
// =====
const LoginScreen = ({ onLoginSuccess }) => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleLogin = async () => {
    if (!email.trim() || !password.trim()) {
      alert('Please enter both email & password! ⚠️');
      return;
    }

    Keyboard.dismiss();
    setIsSubmitting(true);

    try {
      // Simulating API Login request
      await new Promise((res) => setTimeout(res, 1200));

      const fakeJwtToken = `jwt_token_${Date.now()}`;

      // Save securely into Hardware Keystore
      await SecureStorage.saveToken(fakeJwtToken);
      onLoginSuccess(fakeJwtToken);
    } catch (err) {
      alert('Login failed. Check credentials.');
```

```

        placeholderTextColor="#64748b"
        keyboardType="email-address"
        autoCapitalize="none"
        className="bg-slate-950 text-white p-3.5 rounded-xl border border-slate-800 text-sm font-medium"
    />

    <TextInput
        value={password}
        onChangeText={setPassword}
        placeholder="Password"
        placeholderTextColor="#64748b"
        secureTextEntry
        className="bg-slate-950 text-white p-3.5 rounded-xl border border-slate-800 text-sm font-medium"
    />

    <Pressable
        disabled={isSubmitting}
        onPress={handleLogin}
        className="bg-sky-500 active:bg-sky-600 p-4 rounded-xl items-center mt-2"
    >
        {isSubmitting ? (
            <ActivityIndicator size="small" color="#ffffff" />
        ) : (
            <Text className="text-white font-bold text-base">Authorize & Login 🚀</Text>
        )}
    </Pressable>
</View>
</View>
);
};

// =====
// 📱 3. SCREEN B: Home Dashboard (Authenticated Stack)
// =====
const DashboardScreen = ({ userToken, onLogout }) => {
    return (
        <View className="flex-1 px-6 justify-between py-6">
            <View>
                <Text className="text-2xl font-black text-emerald-400 mb-1">Authenticated ✅</Text>
                <Text className="text-slate-400 text-xs mb-6">Hardware Keystore Session Active</Text>

                <View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl gap-3 shadow-xl">
                    <Text className="text-white font-bold text-base">Active JWT Session:</Text>
                    <View className="bg-slate-950 p-3.5 rounded-xl border border-slate-800">
                        <Text className="text-sky-400 font-mono text-xs break-all">{userToken}</Text>
                    </View>
                    <Text className="text-slate-400 text-xs">
                        🔒 This token is stored using hardware-backed AES-256 encryption.
                    </Text>
                </View>
            </View>

            <Pressable
                onPress={onLogout}
                className="bg-red-500/20 active:bg-red-500/30 border border-red-500/40 p-4 rounded-xl items-center"
            >
                <Text className="text-red-400 font-bold text-sm">Sign Out & Destroy Token 🛑</Text>
            </Pressable>
        </View>
    );
};

// =====
// 🚀 4. ROOT APP (Persistent Auth Container)
// =====
const App = () => {
    const [token, setToken] = useState(null);
    const [isInitializing, setIsInitializing] = useState(true);

    // App Launch Lifecycle: Check secure storage for existing session
    useEffect(() => {
        const checkAuthStatus = async () => {
            const savedToken = await SecureStorage.getToken();

```

```

    if (savedToken) {
      setToken(savedToken);
    }
    setIsInitializing(false);
  };

  checkAuthStatus();
}, []);

const handleLogout = async () => {
  await SecureStorage.deleteToken();
  setToken(null);
};

return (
  <SafeAreaProvider>
    <StatusBar style="light" backgroundColor="#090d16" />
    <SafeAreaView className="flex-1 bg-slate-950">
      {isInitializing ? (
        // Splash / Loading Screen while checking hardware keystore
        <View className="flex-1 justify-center items-center gap-3">
          <ActivityIndicator size="large" color="#38bdf8" />
          <Text className="text-slate-400 text-xs font-bold tracking-widest uppercase">
            Verifying Secure Session...
          </Text>
        </View>
      ) : token ? (
        <DashboardScreen userToken={token} onLogout={handleLogout} />
      ) : (
        <LoginScreen onLoginSuccess={setToken} />
      )}
    </SafeAreaView>
  </SafeAreaProvider>
);
};

export default App;

```

🧑 Telugu + English Explanation:

- **expo-secure-store:** Phone lo unna [iOS Keychain](#) & [Android KeyStore](#) hardware chips tho token ni encrypt chestundi. Phone rooted or jailbreak ayina token ni evaru donga-lincha-leru (secure!).
- **Splash / Initializing Screen:** App launch avvagane storage nundi token read chese [600ms](#) time lo [ActivityIndicator](#) chupistham. Idi Login screen nundi Home screen ki unwanted visual flicker raakunda aputundi.
- **Conditional Auth Stack:** `token ? <DashboardScreen /> : <LoginScreen />` pettadam valla, user login or logout avvagane automatic ga screen switch aypothundi.

🔗 Quick Check:

Q: Should you store UI theme preference in [expo-secure-store](#) or [AsyncStorage](#)?

A: AsyncStorage. UI theme is a general setting that does not require hardware cryptographic encryption.

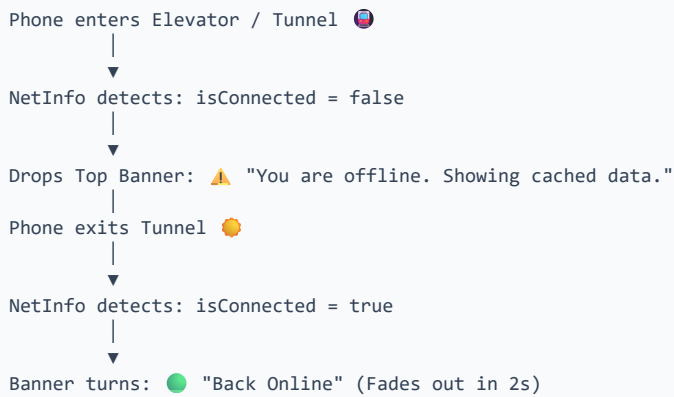
🌐 Level 5.3: Offline Detection & Network Status Banners

1. Mental Model: The Offline-First Mobile Standard

Web browsers lo internet pothe browser blank dinosaur 🦖 screen chupisthundi.

Kani real-world mobile apps (like WhatsApp, Uber, Instagram) lo:

- 1. App **crash avvadu** — UI alage active ga untundi.
- 2. Top lo smooth ga oka animated banner appear avthundi: *"No Internet Connection 🚨"*
- 3. Internet malli connect avvagane banner green ga maaruthundi: *"Back Online ✅"* and 2 seconds lo auto-hide aypothundi.
- 4. Failed API requests automatic ga retry avthayi!



2. The Custom Hook: useNetworkStatus()

```
import NetInfo from '@react-native-community/netinfo';

const useNetworkStatus = () => {
  const [isConnected, setIsConnected] = useState(true);
  const [connectionType, setConnectionType] = useState('wifi');

  useEffect(() => {
    // 1. Subscribe to real-time OS network changes
    const unsubscribe = NetInfo.addEventListener((state) => {
      setIsConnected(state.isConnected && state.isInternetReachable);
      setConnectionType(state.type); // 'wifi' | 'cellular' | 'none'
    });

    // 2. Mandatory Cleanup
    return () => unsubscribe();
  }, []);

  return { isConnected, connectionType };
};
```


Code Example: Interactive Network Status & Offline Banner Simulator

```
import React, { useState, useEffect } from 'react';
import { View, Text, Pressable, FlatList } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// Sample Cached Feed Data
const CACHED_POSTS = [
  { id: '1', title: 'React Native 0.81 Released', tag: 'Mobile', time: '10m ago' },
  { id: '2', title: 'Architecture of Offline-First Apps', tag: 'System', time: '1h ago' },
  { id: '3', title: 'Zero Latency with MMKV Storage', tag: 'Performance', time: '3h ago' },
];

const App = () => {
  // Simulating live network connectivity states: 'wifi' | 'cellular' | 'offline'
  const [networkState, setNetworkState] = useState('wifi');
  const [showReconnectedBanner, setShowReconnectedBanner] = useState(false);

  const isOnline = networkState !== 'offline';

  // Trigger temporary "Back Online" green banner when reconnecting
  const toggleNetwork = (newState) => {
    if (networkState === 'offline' && newState !== 'offline') {
      setShowReconnectedBanner(true);
      setTimeout(() => setShowReconnectedBanner(false), 2500);
    }
    setNetworkState(newState);
  };

  return (
    <SafeAreaView>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950">

        {/* 🚩 1. DYNAMIC TOP NETWORK BANNER */}
        {!isOnline ? (
          <View className="bg-amber-500/90 py-2.5 px-4 flex-row items-center justify-center gap-2">
            <Text className="text-slate-950 font-black text-xs">⚠️ No Internet Connection</Text>
            <Text className="text-slate-900 text-xs font-semibold">(Viewing offline cache)</Text>
          </View>
        ) : showReconnectedBanner ? (
          <View className="bg-emerald-500 py-2.5 px-4 flex-row items-center justify-center gap-2">
            <Text className="text-slate-950 font-black text-xs">🟢 Back Online!</Text>
            <Text className="text-slate-950 text-xs font-semibold">Feed synced with cloud.</Text>
          </View>
        ) : null}

        {/* 🏠 2. MAIN CONTENT AREA */}
        <View className="flex-1 px-5 pt-4 justify-between pb-6">
          <View>
            <View className="flex-row justify-between items-center mb-4">
              <View>
                <Text className="text-2xl font-black text-sky-400">Offline Resilience 🚀</Text>
                <Text className="text-slate-400 text-xs">NetInfo & Network Status Sensor</Text>
              </View>

              {/* Status Badge */}
              <View className={`px-3 py-1.5 rounded-full border ${
                isOnline ? 'bg-emerald-500/20 border-emerald-500/30' : 'bg-red-500/20 border-red-500/30'
              }`} >
                <Text className={`text-xs font-bold uppercase ${
                  isOnline ? 'text-emerald-400' : 'text-red-400'
                }`} >
                  {networkState}
                </Text>
              </View>
            </View>

            {/* Simulated Feed */}
            <Text className="text-white font-bold text-sm mb-3">Recent Feed Articles</Text>
            <FlatList
              data={CACHED_POSTS}
            </FlatList>
          </View>
        </View>
      </SafeAreaView>
    </SafeAreaView>
  );
};
```

```

keyExtractor={(item) => item.id}
ItemSeparatorComponent={() => <View className="h-3" />}
renderItem={({ item }) => (
  <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row justify-between items-center">
    <View className="flex-1 pr-3">
      <Text className="text-white font-bold text-sm">{item.title}</Text>
      <Text className="text-slate-400 text-xs mt-0.5">{item.time}</Text>
    </View>
    <View className="bg-sky-500/10 px-2.5 py-1 rounded-lg border border-sky-500/20">
      <Text className="text-sky-400 font-bold text-[10px]">{item.tag}</Text>
    </View>
  </View>
)}
/>
</View>

{/* 3. INTERACTIVE SIMULATOR CONTROLS */}
<View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl gap-3 shadow-2xl">
  <Text className="text-slate-300 text-xs font-bold uppercase tracking-wider">
    Simulate Network Conditions:
  </Text>

  <View className="flex-row gap-2">
    <Pressable
      onPress={() => toggleNetwork('wifi')}
      className={`flex-1 p-3 rounded-xl items-center border ${
        networkState === 'wifi'
          ? 'bg-sky-500 border-sky-400'
          : 'bg-slate-950 border-slate-800'
      }`}
    >
      <Text className={`font-bold text-xs ${networkState === 'wifi' ? 'text-white' : 'text-slate-400'}`}>
        📶 Wi-Fi
      </Text>
    </Pressable>

    <Pressable
      onPress={() => toggleNetwork('cellular')}
      className={`flex-1 p-3 rounded-xl items-center border ${
        networkState === 'cellular'
          ? 'bg-sky-500 border-sky-400'
          : 'bg-slate-950 border-slate-800'
      }`}
    >
      <Text className={`font-bold text-xs ${networkState === 'cellular' ? 'text-white' : 'text-slate-400'}`}>
        📶 5G Cellular
      </Text>
    </Pressable>

    <Pressable
      onPress={() => toggleNetwork('offline')}
      className={`flex-1 p-3 rounded-xl items-center border ${
        networkState === 'offline'
          ? 'bg-amber-500 border-amber-400'
          : 'bg-slate-950 border-slate-800'
      }`}
    >
      <Text className={`font-bold text-xs ${networkState === 'offline' ? 'text-slate-950' : 'text-slate-400'}`}>
        ✖ Offline
      </Text>
    </Pressable>
  </View>
</View>

</SafeAreaView>
</SafeAreaProvider>
);
};

export default App;

```

 Telugu + English Explanation:

- **NetInfo Listener:** Phone background lo network maarina prathi sari (Wi-Fi off ayina, flight mode pettina) event fire avthundi.
- **Offline Banner:** User internet lekapoyina app open chesi patha cached posts chuskovachu. Screen top lo "*No Internet*" banner petti, user fresh actions (like payments or comments) cheyadaniki try chesthe "*Connect to internet to proceed*" ani clean ga prompt chestham.